Prompt Engineering: Recall Sheet

Structure → Technique → Output → Tools → Cost → Version → Defend

3.1 Prompt Anatomy

Rules in system, data in user, anchor at the end.

- 6 sections: System · Role · Context · Instructions · Examples · Format
- 3 slots: System | User | Tool description
- Start + end = strong · middle = weak
- End-anchor the key rule once; never repeat it 3× mid-prompt
- Multilingual bug? Fix the structure, not the model

Memory: Some Really Clever Interns Edit Files

3.2 Prompting Techniques

Lowest rung first: Zero → Few → CoT → ReAct.

- Few-shot: 3-5 examples fix distribution shift (try before fine-tuning)
- CoT: multi-step tasks only; hurts simple classification
- ReAct: for tools · cap at 5 iterations
- Self-consistency: k=5 votes · 5× cost · high-stakes · temp above 0
- Synthetic PII only in examples

Memory: Zebras Follow Clear Routes

3.3 Structured Outputs

One schema, flatten it, never just retry.

- One Pydantic schema → OpenAI (text_format) + Anthropic (output_config)
- Flatten: extra="forbid" · no top-level oneOf · recursion ≤ 5
- Discriminated union = no silent mis-routing
- Ladder: Validate → Re-prompt → Repair → Escalate (once)
- json_object = any JSON · json_schema strict = enforced

Memory: Very Reliable Robots Escalate

3.4 Function Calling & Tools

Description = prompt. Parallel if independent.

- 5 elements: Purpose · Pre-reqs · Do-NOT-call-if · Use-result · Side-effects
- "Do NOT call if" = biggest accuracy lever
- Independent → parallel · dependent → sequential (say so)
- Cap loop at 5 · alert if avg iterations above 3
- Idempotency key = hash of inputs, never uuid4

Memory: People Prefer Detailed User Stories

3.5 Caching, Batching & Cost

Cache the stable prefix, batch the non-urgent.

- Cache read 0.1× · write 1.25× · TTL 5 min · up to 4 breakpoints
- cache_control at the END of the stable prefix
- Batch = ~50% off · 24h SLA · never for realtime
- History: Summarize / Truncate / RAG, never stuff everything
- Hit rate below 80% → alert · track cost from day 1

Memory: Cheap Bills Tell Dollars

3.6 Prompt Versioning

YAML in Git, tag it, roll out on metric gates.

- Prompt = code = SQL migration (Git, PR, rollback)
- Tags: -v3 immutable · -prod / -canary move
- Rollout 10 → 50 → 100% on metric gates · rollback = canary 0
- CODEOWNERS + CI: compliance and DPO review the diff
- Evals ≠ snapshots (need both) · store read-only in prod

Memory: Git Tags Control Rollouts

3.7 Prompt Injection

Data ≠ instructions: Filter → Spotlight → Validate → Sandbox.

- 3 families: Direct · Indirect (RAG) · Tool
- 5 attacks: Override · Exfiltration · Role-swap · Prompt-leak · RAG
- Spotlight untrusted input · random per-request delimiters
- Side-effect tools need an HMAC confirm token (backend, single-use)
- OWASP LLM01 · 05 · 06 · 08 · never rely on one layer

Memory: Fox Sees Only Safe paths

ALL Golden Rules

If you remember nothing else:

- Structure before cleverness: 6 sections
- Lowest rung that works: Zero → Few → CoT → ReAct
- Validate and repair, never "just retry"
- Cap loops at 5 · idempotency key from inputs
- Cache the stable prefix · batch the non-urgent
- Version every prompt · layer every defence